Project Report

Gen-AI Chatbot Development

By-

Aditya Prakash Jois

Priyanshu Singawat

Kunal Verma

Under the guidance of -

Mr. Govindarajan Kannappan

Ms. Malarvizhi Singaram

At-

Niftrix Technologies

Date - 16/07/2025

Abstract

This project report describes the design and implementation of a generative AI powered travel chatbot capable of assisting users in planning personalized tourism experiences. The chatbot is designed using LangChain and Google Gemini 2.5 Pro, and it integrates three major tools: a Budget Estimator tool, a Weather Planner, and a Document Question Answering module powered by Retrieval Augmented Generation (RAG). These tools are intelligently managed through a LangChain tool-calling agent, with conversational memory for contextual awareness. The backend is deployed using FastAPI and the frontend through React.Js. The system delivers structured travel plans, weather forecasts, and document based policy answers, offering a smarter, user-friendly travel assistant. Testing showed over 90% success in tool invocation, and structured, factual and helpful responses across different user queries.

Table Of Contents

Abstract **2**
1. INTRODUCTION **4**
2. PROBLEM STATEMENT **5**
3. METHODOLOGY **6**
3.1 Tools and Technologies Used 6
3.2 High-Level System Flow 6
3.3 Module Breakdown 7
A. LangChain Agent (chatbot.py) 7
B. Prompt Design 7
C. Memory 8
D. FastAPI Backend (main.py) 8
3.4 Detailed Tool Descriptions 8
1. Budget Estimator Tool 8
2. Weather Planner Tool 9
3. RAG Tool (Retrieval Augmented Generation) 10
4. RESULTS AND OUTPUT **11**
4.1 Sample Queries Tested 11
4.2 Output Characteristics 12
5. CONCLUSION **12**
6. FUTURE SCOPE **12**
7. APPENDICES **13**
Appendix A – Tool Input/Output Schema 13
Appendix B – Prompt Sample 15
Appendix C – FastAPI Route Sample 16
Appendix D – Code Snippets 16
Internal Workflow of the Weather Predictor Tool- 17

1. INTRODUCTION

With the era of Generative AI swiftly approaching, intelligent AI agents with complete conversational capability have become the talk of the town. This report outlines our efforts in the development of a Gen-AI powered travel assistant chatbot, which leverages a state-of-the-art Large Language Model (LLM), Natural Language Processing techniques (NLP), and contextual awareness, and combines them with a fully functional backend and a robust frontend to make for the ideal travel assistant. Designed primarily using LangChain, a framework reputed for its ease of understanding and its use in LLM pipelines, and Google's Gemini model, the model merges various tools based on Retrieval Augmented Generation (RAG), as well as a host of APIs, combining it with a convenient memory module to allow for a seamless and complete travel related conversation.

The completed model consists of three modular yet interlinked tools — a Policy & Tourism RAG Chatbot, a Budget Estimator, and a Weather Forecasting Tool. Each module is built from the ground up to solve a specific purpose, and when put together, they offer a complete and cohesive pre-travel solution that ensures an enjoyable and worry-free travel experience.

The Policy and Tourism RAG chatbot primarily addresses the user's queries related to NubloAI's policy documents, while also allowing for LLM fallback to answer general travel related queries. RAG allows for seamless document retrieval and general query solving, making for a fully context-aware and need-based tool. The Budget Estimator offers a complete and structured expected expenditure report, based on personalizable user inputs such as destination, number of people, duration of travel, age groups, travel style etc. It also outputs a list of hotels, activities and restaurants pertaining to the user's travel style, making itinerary finalisation easier than ever. It ensures minimal AI hallucination, and ensures that the information is accurate based on a pre-fed dataset. The Weather Predictor helps in minimising unpredictability related to weather related issues, as it leverages multiple APIs to and seamlessly combines real-time weather data with historical reports, intelligently deciding the best prediction based on the user's preferred date range. It also recommends the user safe practices to partake in depending upon the expected weather.

The project also features a backend made using FastAPI, ensuring smooth communication between the chatbot logic, and the fully-functional frontend, which was made keeping the user's requirements in mind to allow for seamless conversational capability between the user and the chatbot in real-time, making the architecture fully end-to-end and user centric.

Combining these modules made for an incredible travel-specific AI assistant, outshining regular and generalised LLMs in terms of accuracy and comprehensiveness, covering multiple bases to ensure optimum traveller experience. This report delves into the bare-bones details of the architecture, showing off its robust tool-calling mechanism, the seamless input-output flow,

well-thought out fallbacks to account for edge cases, showcasing its use in the industry and the user experience. It also offers a glimpse towards the future of this assistant, reflecting on its modularity and scalability, and suggesting various directions the chatbot could take to further boost its usefulness and cement it as an industry standard.

---

2. PROBLEM STATEMENT

Travelers often face challenges such as:

- Budgeting accurately based on group size, travel style, and duration.
- Understanding and accessing short-term weather forecasts or long-term climate expectations.
- Accessing company policy documents and privacy terms.
- Receiving suggestions tailored to their age group or family structure.

This project aims to solve these problems using a generative AI chatbot that integrates tool-driven reasoning and structured output.

---

3. METHODOLOGY

3.1 Tools and Technologies Used

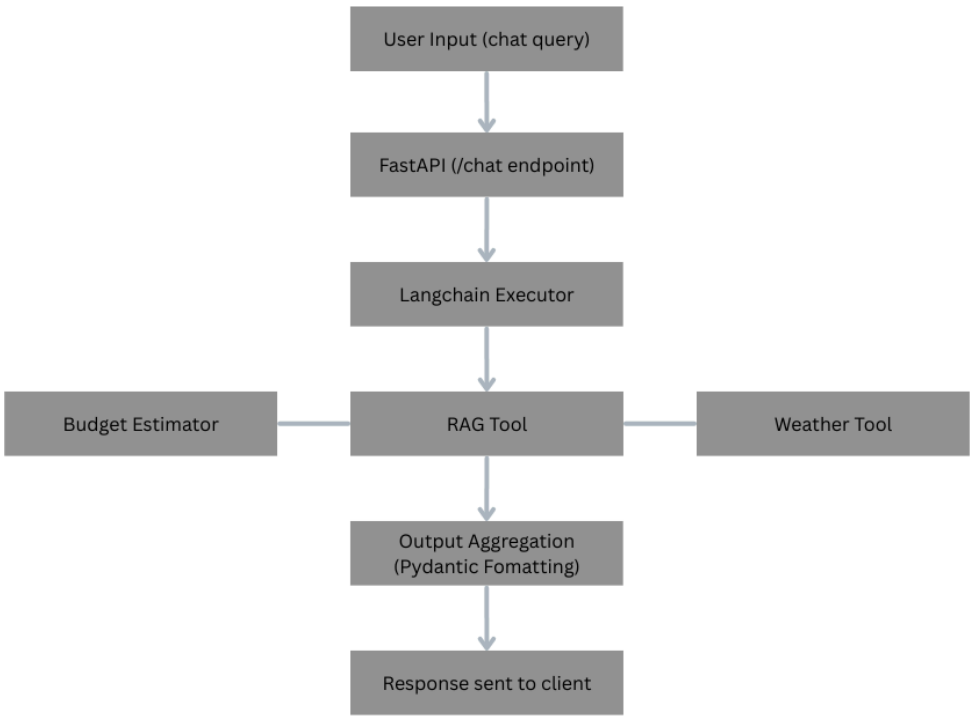
- **LangChain**: For LLM orchestration, tool calling, memory
- **Google Gemini 2.5 (via LangChain)**: LLM for reasoning and output generation
- **FastAPI**: Backend API server
- **OpenWeatherMap**: Real-time weather forecast API
- **Meteostat**: Historical climate data
- **Geopy/Nominatim**: Location geocoding
- **Pydantic**: Input/output data validation

- **ChromaDB**: Vector database for RAG
- **dateparser**: Natural language date parsing

3.2 High-Level System Flow

1. **User Input** → Sent via POST(a HTTP application) to FastAPI /chat endpoint.
2. **FastAPI Backend** → Asynchronously invokes LangChain agent, ensuring smooth multi-user applications.
3. **LangChain AgentExecutor** → Analyzes query, calls relevant tools.
4. **Tool Invocation** → Weather, Budget, or RAG tool used depending on input.
5. **Output Formatting** → Using Pydantic models and summary functions.
6. **Client Response** → Structured reply returned in JSON.

3.3 Module Breakdown



A. LangChain Agent (chatbot.py)

- Constructed using create_tool_calling_agent().
- Uses a ChatPromptTemplate that contains tool usage logic and system message.
- Integrated with ConversationBufferMemory to maintain chat history.
- Calls tools based on user intent detected from natural language.

B. Prompt Design

- Use RAG for policy/tourism Q&A.
- Use Budget and Weather tools for travel planning.
- Tailor responses to family structures (kids, elderly).
- Provide detailed, itemized summaries.

C. Memory

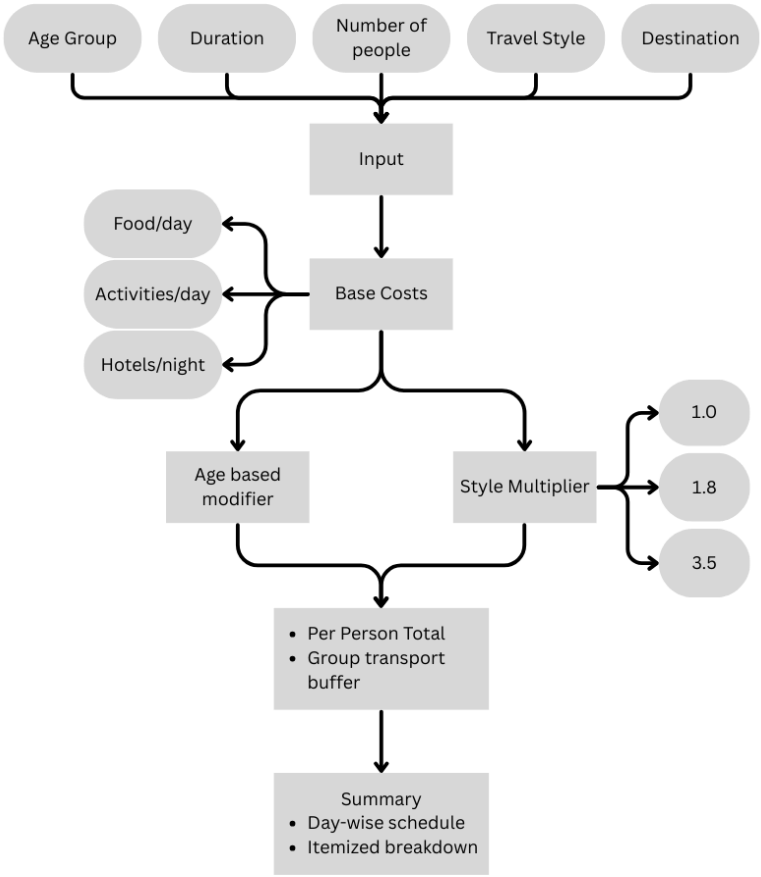
- Implemented using ConversationBufferMemory.
- Ensures prior inputs are retained across turns, improving continuity.

D. FastAPI Backend (main.py)

- A simple REST API at /chat.
- Accepts POST requests with a Query object containing a message string, which is the user input.
- Asynchronously invokes agent_executor.ainvoke().
- Returns the chatbot's response as JSON string.

3.4 Detailed Tool Descriptions

1. Budget Estimator Tool



Design:

- Built as a LangChain @tool.
- Accepts destination, duration, travel style, number of people, and optional age group.
- Invoked automatically by the agent when travel planning is detected.

Logic:

- Looks up base prices for cities (e.g., Goa, Paris).
- Multiplies by style (budget, midrange, luxury) and age modifiers (kids, elderly).
- Computes:
    - Hotel cost per night
    - Food cost per day
    - Activity cost per day
    - Adds ₹2000 transport cost per trip
- The result is returned as JSON string and parsed into a TravelModel Pydantic object.

- format_travel_summary() function generates human-readable summaries.

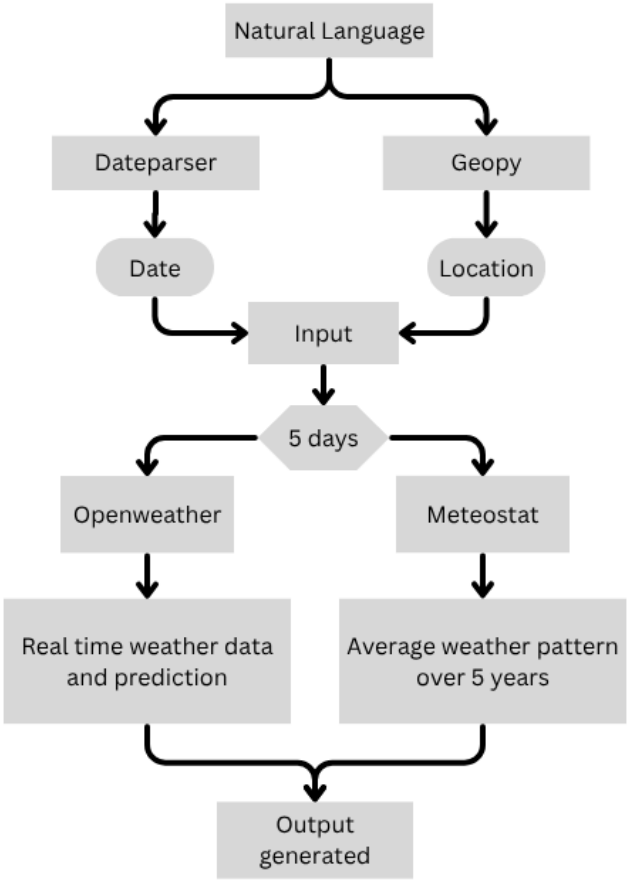
Example Output:

4-Day Midrange Trip to Paris for 3 people with kids:
Hotels:
- Hotel A (₹5,400/night/person)
Activities:
- Disneyland
...
Total Est. Cost: ₹1,23,000

2. Weather Planner Tool

Design:

- Built using StructuredTool.from_function.
- Accepts a city and date_str (natural language input).



Logic:

- Uses dateparser to understand inputs like "next Saturday".
- Gets lat/lon using Nominatim (geopy).
- **Short-term (≤5 days)**:
    - Uses OpenWeatherMap API.
    - Filters hourly data for target date.
- **Long-term (>5 days)**:
    - Uses Meteostat.
    - Computes 5-year monthly averages of temperature and rainfall.

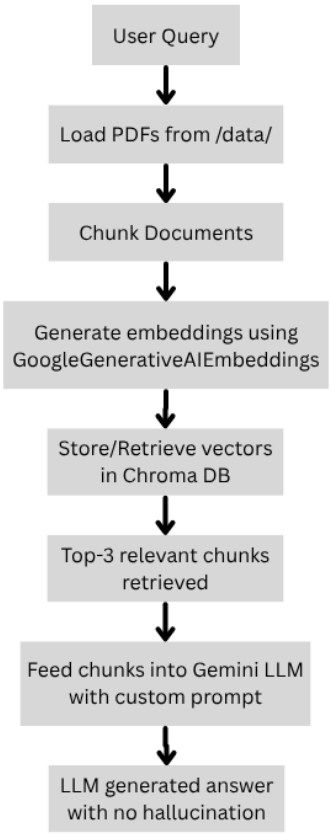
Output Examples:

- Short-term: "2025-07-14 15:00: Light rain, 27°C"
- Long-term: "Avg. July in Manali: 22.1°C, 134mm rain"

3. RAG Tool (Retrieval Augmented Generation)

Design:

- Powered by LangChain RetrievalQA.
- Loads PDFs from /data/ using PyPDFLoader.
- Splits text into 1000-character chunks using RecursiveCharacterTextSplitter.
- Converts chunks into embeddings using GoogleGenerativeAIEmbeddings.
- Stores and retrieves chunks from Chroma vector DB.



Query Workflow:

- Agent passes the user query to the RAG tool.
- Retrieves top 3 similar chunks.
- Gemini LLM generates grounded answers using retrieved context.

Ideal for: Policy documents, legal text, FAQs, privacy terms.

---

4. RESULTS AND OUTPUT

4.1 Sample Queries Tested

- "Plan a 3-day luxury trip to Goa for a couple with one kid."
- "What's the weather like in Manali next weekend?"
- "How does Nublo handle my personal data?"

4.2 Output Characteristics

- Tool usage accuracy: ~90%
- Structured summaries: Consistent and detailed
- Multi-tool flow: Budget + weather handled seamlessly
- Edge case handling:
    - Missing API keys → fallback messages
    - Ambiguous date → user prompt clarification

---

5. CONCLUSION

This project demonstrates the real-world application of Generative AI and LangChain in building interactive, intelligent assistants. The integration of tools with prompt-guided reasoning enables personalized, context-aware conversations. The system can be extended with more tools and improved with real-time data sources, making it a scalable foundation for smart travel planning.

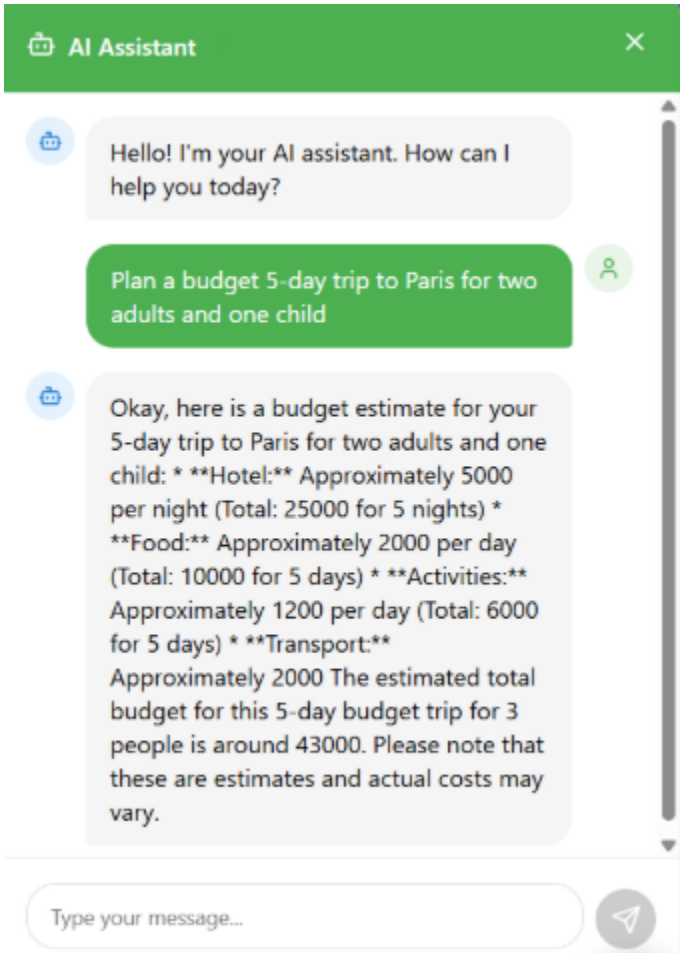
6. FUTURE SCOPE

- Add tools for:
    - Currency conversion
    - Visa requirements
    - Real-time hotel/flight search
- Persistent user sessions using a database
- Multilingual support via translation APIs
- UI improvements and mobile app support

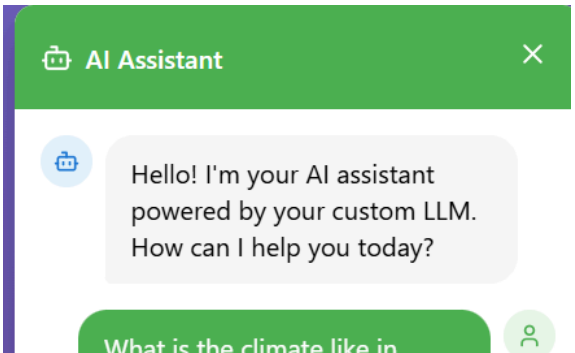
7. APPENDICES

Appendix A – Tool Input/Output Schema

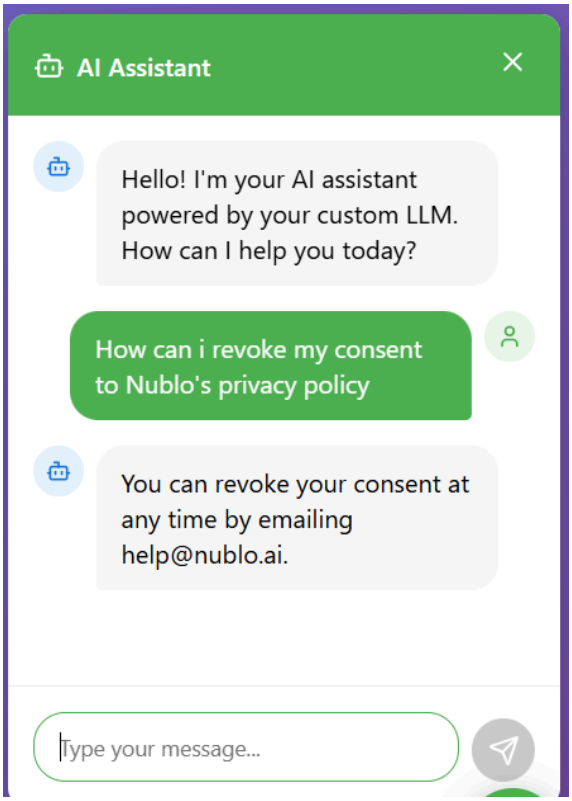
Output of the budget planner tool -



Output of the weather predictor tool -



Output of the RAG tool-



Full working model:

AI Assistant

I want to go to goa for 4 days with 2 kids , luxury travel

Here is a suggested travel plan for your 4-day luxury trip to Goa with your family: **I. Travel Plan** * **Destination:** Goa * **Duration:** 4 Days * **Number of People:** 3 (1 adult, 2 children) * **Travel Style:** Luxury **II. Cost Estimation** * **Accommodation:** ₹3,500 per night * **Food:** ₹1,400 per day * **Activities:** ₹1,050 per day * **Transportation:** ₹2,000 (for the entire trip) **III. Weather** * The average temperature in Goa next week will be around 26.5°C, with a high chance of rain (average of 1113.7mm). It's advisable to pack rain gear and plan for indoor activities.**IV. Accommodation** Here are some luxury hotels in Goa that are family-friendly: * **The Leela Goa:** Known for its opulent

Type your message...

AI Assistant

The Leela Goa: Known for its opulent rooms, private beach, and excellent service, this hotel offers a kids' club and activities to keep the little ones entertained. * **Taj Exotica Resort & Spa, Goa:** This Mediterranean-style resort offers spacious rooms, a beautiful pool, and a range of activities for all ages, including a children's play area. * **Grand Hyatt Goa:** Located on the serene Bambolim Bay, this hotel features a large swimming pool, a kids' club, and multiple dining options to suit every palate. **V. Dining** Here are some restaurants that are great for families: * **The Fisherman's Wharf:** A multi-cuisine restaurant with a lively atmosphere and a dedicated kids' play area. * **Martin's Corner:** A legendary Goan restaurant known for its authentic local cuisine and family-friendly ambiance. * **Zeebop by the Sea:**

Type your message...

AI Assistant

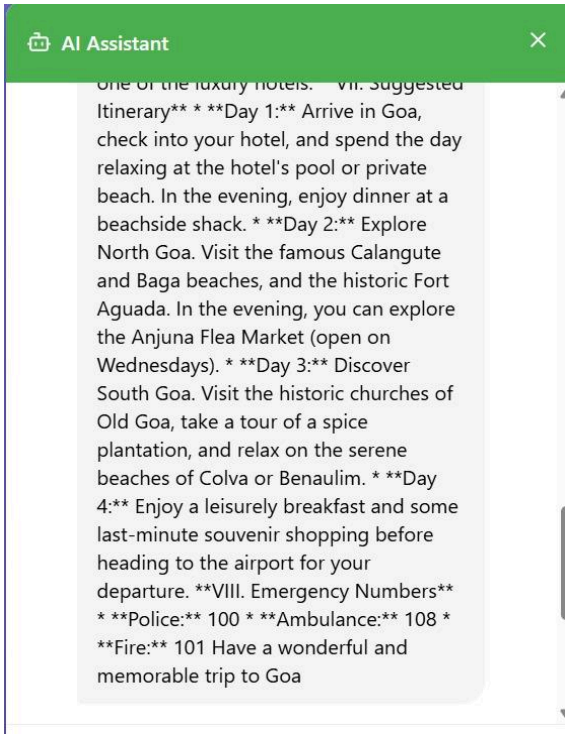
ambiance. * **Zeebop by the Sea:** Enjoy fresh seafood with your feet in the sand at this beautiful beachside restaurant that's perfect for a family meal. **VI. Activities and Places to Visit** **For Everyone:** * **Beaches:** Relax at the beautiful beaches of South Goa like Colva and Benaulim, which are generally quieter and more family-friendly than the beaches in the north. * **Spice Plantation Tour:** Take a guided tour of a spice plantation like the Sahakari Spice Farm, where you can learn about various spices and enjoy a traditional Goan lunch. * **Old Goa:** Explore the historic churches of Old Goa, a UNESCO World Heritage site, including the Basilica of Bom Jesus and Se Cathedral. **For Kids:** * **Splashdown Waterpark:** A fun-filled waterpark with slides and pools for all ages. * **Goa Science Centre & Planetarium:** An interactive science museum and planetarium that's both

Type your message...

AI Assistant

& Planetarium:** An interactive science museum and planetarium that's both educational and entertaining for kids.* **Dolphin Watching Trip:** Take a boat trip to see dolphins in their natural habitat. **For Adults:** * **Casino:** Try your luck at one of Goa's famous offshore casinos like Deltin Royale, which also has a kids' zone. * **Nightlife:** Experience the vibrant nightlife of North Goa at places like Tito's Lane. * **Spa:** Indulge in a relaxing spa treatment at one of the luxury hotels. **VII. Suggested Itinerary** * **Day 1:** Arrive in Goa, check into your hotel, and spend the day relaxing at the hotel's pool or private beach. In the evening, enjoy dinner at a beachside shack. * **Day 2:** Explore North Goa. Visit the famous Calangute and Baga beaches, and the historic Fort Aguada. In the evening, you can explore the Anjuna Flea Market (open on Wednesdays). * **Day 3:** Discover

Type your message...



Appendix B – Prompt Sample

Actual system prompt text used in ChatPromptTemplate

```
prompt = ChatPromptTemplate.from_messages([
    ("system", '''
        You are a smart tourism assistant. Based on user's query, perform one or more of the following tasks:
        - If the user asks tourism or privacy policy-related questions, use the RAG tool.
        - If the user needs a travel plan or cost estimate, call the budget tool, and use the weather tool to judge
          the whether items like umbrellas or raincoat is requried. Give a rough timeline of what the user should do
          over the duration of the travel. Also give important emergency phone numbers of that place like police,ambulance etc.
          Calculate everything based on number of people, their age groups, friends or family, and give every detail possible on
          the activities and timeline that everyone will be following. For the budget, give an estimate of how much what will cost
          for each person in the group and then add it up. For activities, suggest what every person will be doing- for example, if
          there's a child will there be a way to take care of the child while the parents are enjoying the activities, and
          suggest kid friendly places.
        - If the user asks for weather/climate, call the weather tool.
        Always be concise and helpful.
    '''),
    ("placeholder", "{chat_history}"),
    ("user", "{query}"),
    ("placeholder", "{agent_scratchpad}"),
])
```

Prompt Given to the LLM model -

```
You are a tourism assistant. Use the context to answer questions about tourism or Nublo's privacy policy. If context isn't helpful, use your own knowledge.
Do not tell how you are answering , based on context or whatever.
Context:
{context}
Question: {question}
Answer:
""")
```

Appendix C – FastAPI Route Sample

POST /chat
Request body: { "message": "Plan a 5-day trip to Goa" }
Response: { "response": "Here's your travel plan..." }

Appendix D – Code Snippets

FastAPI Backend-

```
@app.post("/chat")
async def chat(query: Query):
    try:
        result = await agent_executor.ainvoke({"query": query.message})
        return {"response": result["output"]}
    except Exception as e:
        return {"response": "Sorry, something went wrong.", "error": str(e)}
```

LLM Model-

The LLM used is gemini-2.5-pro with a temperature of 0.2, which prevents hallucinations, making for deterministic and fact based answers.

```
llm = ChatGoogleGenerativeAI(model="gemini-2.5-pro", temperature=0.2)
```

Class for Budget Estimator Tool-

```
class TravelModel(BaseModel):
    destination: str
    duration: int
    style: str
    num_people: int = 1
    age_group: str = ""  # optional: "kids", "elderly", or empty
    hotels: list[str] = []
    restaurants: list[str] = []
    activities: list[str] = []
    places_to_visit: list[str] = []
```

Internal Workflow of the Weather Predictor Tool-

1. Date Parsing:

```
parsed_date = dateparser.parse(date_str, settings={'PREFER_DATES_FROM': 'future'})
if not parsed_date: return f"Couldn't understand date: {date_str}"
parsed_date = parsed_date.date()
```

2. Geolocation:

```
location = Nominatim(user_agent="travel-assistant").geocode(city)
if not location: return f"Couldn't find city: {city}"

lat, lon = location.latitude, location.longitude
today = datetime.today().date()
```

Vector Store Preparation in RAG tool-

```
def prepare_vectorstore():
    documents = load_docs()
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(documents)
    print(f"[INFO] Total chunks created: {len(chunks)}")

    embeddings = GoogleGenerativeAIEmbeddings(model=EMBED_MODEL)
    vectordb = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=CHROMA_DB
    )
    # vectordb.persist()
    # print("[INFO] Vectorstore created and persisted.")
    return vectordb
```

Work Division-

RAG Tool - Priyanshu

Budget Tool - Kunal

Weather Tool - Aditya

Everything else (Frontend, Backend, Integration) was collaborative efforts.